第五部分 · 记忆的检索与利用机制

检索到的记忆怎样进入模型推理、规划与行动？

从“找到过去的信息”到“达成可验证的正确行动”

Memory → Context → Reasoning → Planning → Action → Verification

> 找到记忆，不等于正确理解。
> 生成计划，不等于执行成功。
> 工具返回，不等于目标已经达成。

零基础理论课 · 16张原理机制图 · 12道理论题与详解

v1.0 | 2026-10-19 | 仅理论，无实验

《智能体记忆系统：从零基础到科研理解》 · 第24/32讲 · 第五部分：记忆的检索与利用机制

副标题：从“找到了过去的信息”到“做出可核验的正确行动”

本讲的核心问题： 记忆检索系统已经把全部正确的历史片段送回来了，智能体却仍然做错事。这种错误究竟出在什么地方？模型是如何接触到记忆的？为什么一段文字形式的计划不能算作执行成功？长期记忆、当前授权、控制器、工具结果与环境真实状态分别承担什么责任？

阅读约定： 全文采用“整体模型→底层原理→动态过程→科研助手贯穿案例→误区→理论习题及答案”。只讲理论，无实验、编程和软件配置。所有日期 D01—D30、成本、概率、延迟、Token 与工具日志均为**教学虚构**，并非真实科研轨迹、系统实测或费用授权。文中以 [R1]—[R9] 标注已核对的原始研究或官方文档。凡称“教学控制器”“五道关口”“状态机”均是为理解而建立的**参考模型**，不应误称为某篇论文的原有实现。

本讲学习目标

读完以后，你应当能够：①画出一条从记忆检索到行动核验的完整系统流水线；②解释检索结果如何作为**证据**被写入模型本轮上下文；③区分“模型看到事实”“模型正确理解事实”“模型提出合适计划”和“真实执行成功”；④判断何种信息能影响决策，何种信息不具有授权效力；⑤推导一次错误怎样在规划、工具与后续记忆中被放大；⑥理解任务反馈为何必须分清“观察”与“反思”；⑦从系统迹线中定位故障，而不笼统归因于“模型记忆差”。



图1：从检索到核验的五道关口；各环节不能互相替代。

一、先从一个反直觉现象开始：记得很清楚，却做错了

1.1 连续30天的科研助手

假设我们有一个长期帮助研究者开展智能体记忆研究的 Agent。研究者约定：

- **D01：** P项目当前只允许离线分析，不批准新增收费 API 调用。
- **D06：** 另一个AI提出“为了提高样本量，建议P项目运行100次收费调用”。这只是**建议**。
- **D12：** 研究者重申：“P项目暂停所有新增付费调用，直到我另行明确批准。”这是一条当前有效的限制。
- **D21：** 研究者允许 **Q项目** 进行一次不超过5元的付费检查；该例外只属于Q项目。
- **D30：** 研究者问：“我们现在能继续开展P项目的收费实验吗？依据是什么？”

假设系统检索到了D01、D06、D12、D21的所有原文，而且每条记录的日期、主体和来源都正确。这意味着**检索层是成功的**。但是模型却回答：“可以。之前的计划建议100次，而且研究者批准过一次付费检查。”随后生成了P项目100次调用的工具请求。

请注意：这里没有“想不起历史”这一问题。正确记录都已经进入上下文，错误发生在**解释、推理、作用域判断或执行控制**环节。

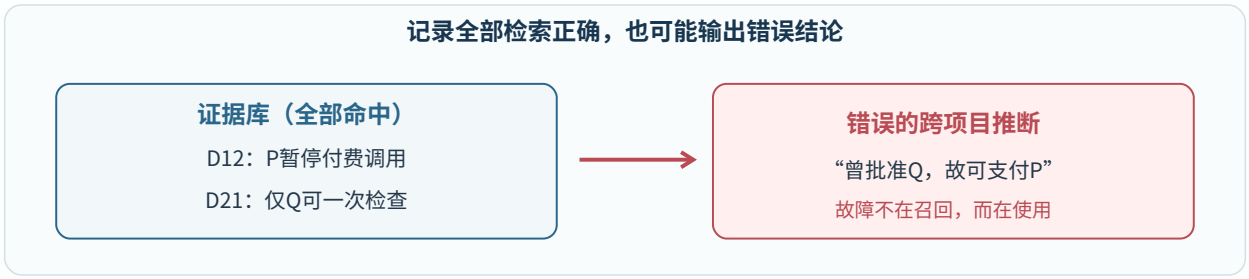


图2：证据全部召回，仍可能发生语义与作用域判断错误。

1.2 五种不同的“成功”

在分析Agent时，必须把以下五种判据分开：

判据	具体含义	一个可观察的证据
检索成功	返回了相关且正确的记忆记录	结果集中确实有D12原文
阅读/理解成功	模型从记录中抽取了正确语义	判断D12是对P项目的限制
决策/规划成功	提出的行动满足目标和约束	给出“P仍需等待批准”的结论
执行成功	经过控制器允许，工具调用完成	有合法调用和真实工具回执
任务成功	环境中的目标状态确实达成	结果经独立状态或评分器核验

在只需回答问题时，后两项未必涉及外部工具；但在能行动的Agent中，它们是独立且不可省略的环节。**检索成功率不能直接当成任务成功率。**

二、记忆究竟怎样“进入”模型：从检索结果到上下文

2.1 模型并没有一个外置的“记忆感官”

传统LLM通常通过**本轮输入的Token序列**接收相关信息。外部记忆库可能保存数百万条记录，但模型这一次只会读取**被上层软件选中并序列化到输入上下文中的部分**。检索工具拿到一条记录，本质上是上层程序将它转换为类似下面的信息：

> 【记忆 E12】项目：P；发生时间：D12；来源：研究者直接陈述；主张：“P项目暂停所有新增收费API调用，待研究者另行明确批准”；状态：当前有效；原文指针：event_D12。

这种结构化文本是**给模型看的证据**，不是模型内部权重突然变化，也不是“记忆数据库融进了神经网络”。在具体框架里，序列化可能是消息、JSON、XML样式标签或工具结果，它们只是不同载体。

2.2 一次高质量的上下文组装，需要分清四层

1. **任务层**：用户到底要回答什么，目标对象是P还是Q?
2. **当前约束层**：谁有权决定费用、哪条限制正在生效?
3. **证据层**：E01、E06、E12、E21的原始内容、时间、来源、状态和引用指针。
4. **执行接口层**：允许使用哪些工具、工具参数的形式、在什么条件下可以调用。

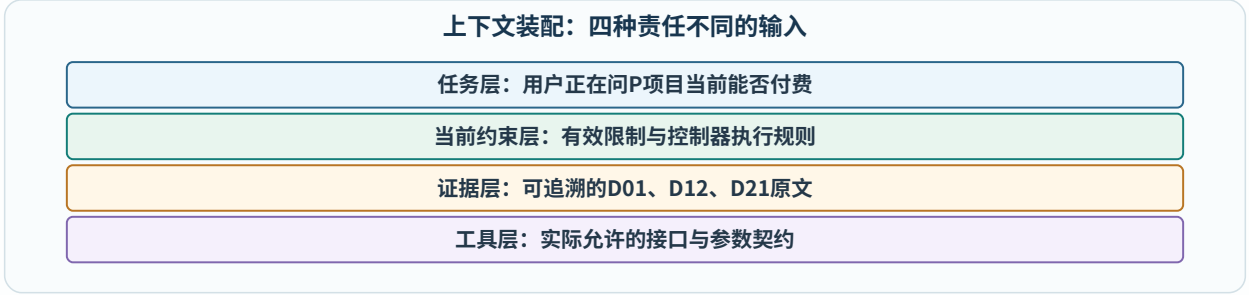


图3：上下文组装的四层职责——任务、约束、证据、工具。

系统可把过去记忆展示为“证据材料”，但不能把来源不明的历史文本自动升级成高优先级系统规则。比如，E06的AI建议即使语言极其明确，也不具有修改研究者费用限制的权限。**消息的权威性来自可信身份、适用范围和授权机制，不来自它的相似度或排版位置。**

2.3 “装进上下文”仍不保证“正确使用”

Transformer会利用注意力等运算在不同输入位置之间传递信息。但注意力的权重不是“事实是否可信”的判决器。模型可能更关注较近的文本、语气肯定的指令、较长的建议，或者把有条件的例外不当地推广。Liu等在《Lost in the Middle》中观察到部分模型的长上下文任务性能随相关信息在输入中的位置而变化；这说明**模型可接收信息的容量与有效使用信息的能力并非一回事**。[R4]

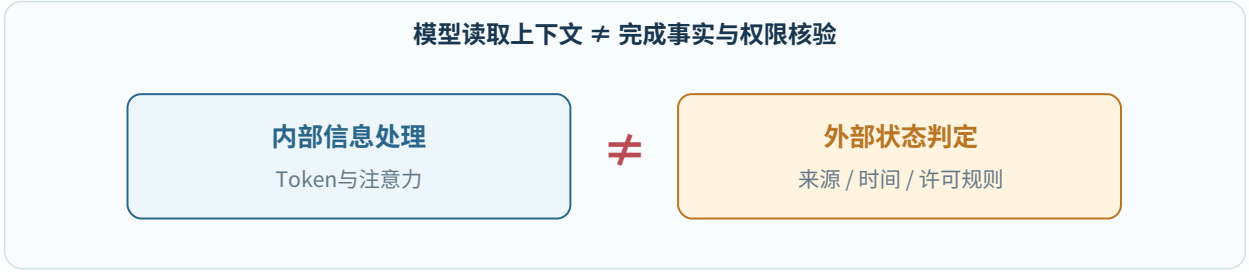


图4：内部信息利用与外部事实/权限核验是两种不同职责。

> **重点：** 更长上下文、更高Top-K、更多引用通常能提高“看到信息的机会”，但并不自动提供时间推理、授权判断、约束满足或工具状态核验能力。

2.4 参考的Token装配预算

假设课堂模型本轮允许使用8,192 Token，预算如下。数字仅用于计算，不代表任何真实模型的固定协议：

部分	Token预算
系统与应用控制说明	1,280
本轮问题、任务状态	1,200
经过挑选的记忆证据	1,800
工具说明与近期观测	680
预留输出	1,800
合计	**6,760**
剩余容量	**1,432**

虽然剩余1,432 Token，**也不意味着应该再塞入1,432 Token的记忆**。过度加入无关信息可能造成注意力竞争、错误的跨项目联想或延迟增加。这个预算例子从“物理能放多少”引出“对本轮决策真正需要多少”。

三、正确使用记忆的第一关：分清事实、建议、约束与行动许可

3.1 同一句话可能具备完全不同的认识地位

从语义检索系统看，“建议进行100次API调用”和“批准进行100次API调用”在主题、数量、技术词汇上非常相似；但对执行控制而言，它们完全不同。需要至少问四个问题：

- **谁说的（Who）？** 研究者、外部论文、AI助手还是工具?
- **说的是什么（What）？** 观察、建议、假设、决定、回执还是解释?
- **在什么条件下有效（Scope / When）？** P还是Q、D12之后还是之前、预算多少、何时失效?
- **究竟支持什么行动（Authority）？** 是否明确允许收费、删除文件、提交论文等具体操作?

来源身份	主张或观测	对行动的效力
AI助手	D06付费建议	只是建议
研究者	D12暂停P	当前限制
研究者	D21批准Q一次	只适用于Q
工具返回	D09 HTTP504	只是观察

图5：同一主题下的四类消息，授权效力并不相同。

3.2 “更近的记忆”不一定更有权威

D21比D12更近，但它只允许Q项目一次付费检查。检索排序器可能把D21排在前面；如果模型只使用“用户最近批准过费用”这个粗粒度摘要，就会错误允许P项目收费。正确推断是：**P和Q属于不同作用域**，D21不构成对P限制的替代。只看日期而不看对象，是典型的状态合并错误。

3.3 强相关不等于强证据

对于“P项目是否已获得收费许可”，E06的主题相似度可能高于E12，但E06是AI建议。可以用一条简化的硬约束规则理解：

> 在做当前许可判断时，先排除**项目范围不匹配**、**非有效日期**、**来源身份不匹配**及**未经批准的主张**；对于剩余的有效证据，才讨论相关性和时间排序。

这是一种工程上的“先资格、再排序”设计思想，并非声称所有记忆系统都内置这一公式。

四、正确使用记忆的第二关：从“已知事实”形成“有约束的计划”

4.1 事实本身不是计划

例如“D12禁止P收费调用”是一条状态事实。计划需要回答：现在要实现什么目标？需要执行哪些步骤？有哪些前置条件？每一步由哪个实体执行？如何判断真正完成？

同一事实可以导出不同计划：

- 用户问“P现在能不能付费？”——直接回答限制，给出处；**无需调用收费工具**。
- 用户问“先做不收费的资料检查”——设计离线读取和总结的计划；不应偷偷付费。
- 用户问“如果未来批准收费，需要哪些步骤？”——提供**条件化方案**；方案不是执行授权。



图6：从事实到计划要额外检查目标、前置条件与候选动作。

4.2 三个经常被混淆的层次

任务目标（Goal）是希望最终达到的状态，比如“确认实验是否符合约束”。**计划（Plan）**是达到该目标的候选步骤，比如“读取项目状态→核对限制→需要时询问负责人”。**动作（Action）**是实际执行的一个可验证操作，比如对项目存储库进行一次读取。模型输出一段计划，只代表给出了候选方案，并不等于工具已运行。[R1]

4.3 规划应考虑前置条件、分支与终止条件

一个可靠的教学规划结构至少包括：

字段	为什么需要	P项目示例
目标	说明最终判断对象	判断能否新增收费实验
前置条件	限制执行前必须成立的事实	存在P作用域的明确许可
候选动作	当前允许考虑的下一步	读取权威状态 / 请求批准
失败或分支	遇到证据不足时怎么办	不执行收费，反馈缺口
成功判据	什么算做完	状态查明并给出证据
终止条件	防止无限试错或越界	权威状态拒绝或无新证据

规划过程可以是文本步骤、显式状态机或其他算法，并非所有Agent都使用一个专门的“规划器”模块。这里讲的是**概念职责**，不是规定唯一工程架构。

4.4 为什么有时候记忆能帮助规划，有时候反而干扰

记忆可以提供以往成功操作、历史约束和失败经验，减少重复探索。Generative Agents利用检索与反思辅助规划行为，是这个思路的代表之一。[R2] 但是，过去成功的方案并不保证适用于当前环境；若工具版本变化、权限改变、任务目标变化，照搬历史步骤会带来**策略迁移错误**。

五、正确使用记忆的第三关：控制器与模型必须有明确职责边界

5.1 模型建议行动，不等于系统必须执行

在可使用工具的Agent中，应区分两类角色：

- **语言模型**：从当前上下文提出判断、计划、候选工具名称和参数。
- **控制器/执行器**：依据应用实现中的工具清单、权限、参数约束、预算、真实状态等，决定是否实际执行。

这与ReAct提出的推理—行动—观察交替过程相关，但本讲的“硬性授权门”是工程教学参考设计，不是说ReAct论文已经解决了所有许可与验证问题。[R1]



图7：控制器将“模型想做”与“系统准许执行”分隔开。

5.2 一次工具调用至少包含五个不同阶段

1. **Proposed**：模型说“建议调用某个工具”，只是候选。
2. **Validated / Allowed**：控制器检查参数、预算和执行权限后允许操作。

3. **Dispatched**：请求真正发往工具或外部环境。
4. **Observed**：工具返回输出、错误码、异常或超时。
5. **Verified**：系统根据权威环境状态、明确的验收规则判断目标是否完成。



图8：工具调用经历提出、验证、发送、观察与验收五个阶段。

这五个阶段可能被不同框架以不同名字或日志表示；但概念边界必须保留。尤其**工具返回“HTTP 200”只能说明所采用的某个通信接口返回了成功响应**，不等于业务目标一定完成；可能只是请求已受理、异步任务排队或部分步骤完成。相反，HTTP 504并不总能证明下游任务绝对没有发生，若涉及不可重复的写操作，简单重试甚至可能导致重复副作用。

5.3 工具调用失败时如何安全继续

遇到“工具超时”时，下一步并不是无条件再试一次。应先区分只读请求和有副作用请求、是否存在幂等键、是否有权威状态查询接口、是否需要人工确认。正确的诊断通常依赖**工具日志和环境状态**，而不是模型对超时的主观解释。[R5]

最小闭环应当包括：产生动作→获得实际回执→读取或核验环境→记录结果与不确定性。在尚未验证环境之前，不能把“我已经成功完成”写成确定的情景记忆。

六、正确使用记忆的第四关：反馈进入后续记忆时必须分层

6.1 观察与反思不能混写

假设D09执行日志显示“HTTP 504”。这只是工具观察。D11智能体写出：“应该是模型推理能力差导致的。”这属于AI解释，不是直接观察。D18工程日志最终显示“请求经过的代理网关超时”，这是带来源的后续诊断。它们应作为不同类型的记录存在，并建立证据关系，而不是把D11猜测默默覆盖成原始事实。

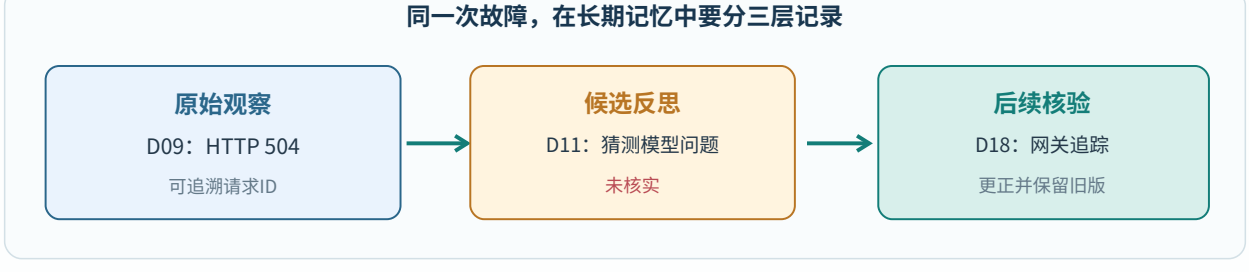


图9：观察、候选推断和经核验的新诊断必须分层保存。

6.2 三层反馈记忆

- **原始轨迹**：用户输入、模型建议、工具参数、实际响应、时间戳、环境可观察变化。
- **经核验的项目状态**：哪些操作完成，哪些未完成，最新限制和有效版本是什么。
- **候选经验/反思**：从多个案例中总结“类似问题先核查网关”。这是可以复用的**经验建议**，仍要受任务与环境约束。

Reflexion展示了智能体通过语言反馈保存反思以改善后续尝试的框架；其研究并未意味着“每条自我反思天然正确”或“模型权重因此自动更新”。[R3]

6.3 错误如何跨多轮固化

一次错误如果未经标记地写入长期记忆，就可能形成自强化循环：错误行动→错误总结→未来检索→下一次错误行动。不能因为下一轮又检索到了这条“经验”，就认定它已经得到多次独立验证。

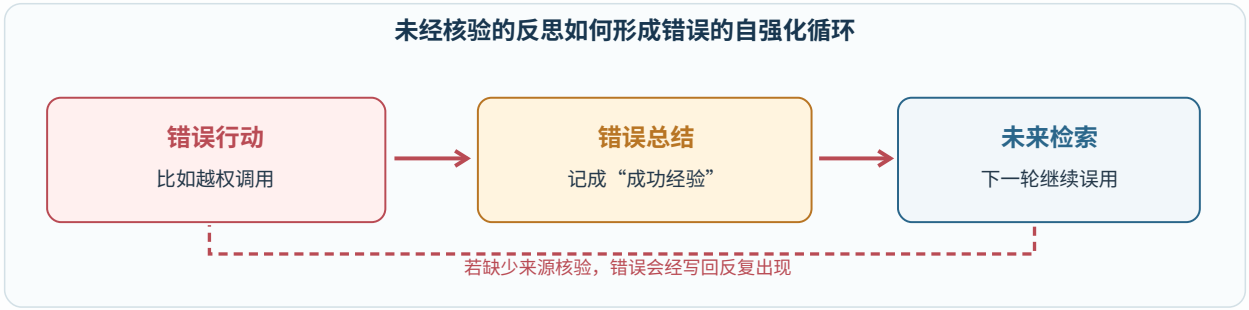


图10：未经核验的反思可能通过长期记忆形成错误循环。

七、完整动态推演：一条从检索到行动的科研助手轨迹

下面是本讲的**完整示范**，所有事件均为虚构。

7.1 第一步：收到用户问题

D30用户问：“P项目今天能开始收费实验吗？请按之前的约定说明。”这里包含四个要点：对象P、当前时间D30、收费这一高影响动作、要求依据。控制器或模型应识别：这不是泛泛的技术建议，而是在问**当前许可状态**。

7.2 第二步：读取权威状态与相关历史

查询过程找到了E01（P离线范围）、E06（AI的100次付费建议）、E12（P暂停）、E21（Q的一次性许可）。适当的证据读取结果必须保留project_id、source_role、valid_time、status和原文引用，不应压缩成“过去讨论过付费，后来批过一次”。

7.3 第三步：建立判定表

证据	来源身份	适用项目	事件性质	对当前P收费的效力
E01 D01	研究者	P	初始范围	不允许新增收费
E06 D06	AI助手	P	候选建议	不构成授权
E12 D12	研究者	P	明确限制	当前仍需遵守
E21 D21	研究者	Q	一次性例外	不能用于P

正确结论： 截至本例D30，P尚未获得新的付费许可。系统可以直接解释此判断，或建议先开展可授权的离线工作；应避免提交P付费工具调用。

7.4 第四步：若用户要求“准备好实验计划”

允许生成待批准的方案：列出API请求上限、预计成本、停止条件与异常处理。**方案草稿**可以保存到项目材料，但需要标注`status=draft`，不能写入 `authorization=approved`。如果之后研究者确实批准了某个特定范围，还要按许可的项目、预算、次数与有效时间执行，而不能照搬E06的100次建议。

7.5 第五步：若系统错误地试图执行

即使模型提出`run_paid_trial(project='P', n=100)`，控制器也应根据可靠的权限来源阻止它。日志应保留：“模型提出候选动作；控制器因P缺少新授权而拒绝；实际工具未被调用”。用户答复要说清“没有执行”，不能凭模型已经写出的工具调用文本假称“测试已开始”。



图11：D01—D30教学时间线，项目作用域优先于简单的近期性。

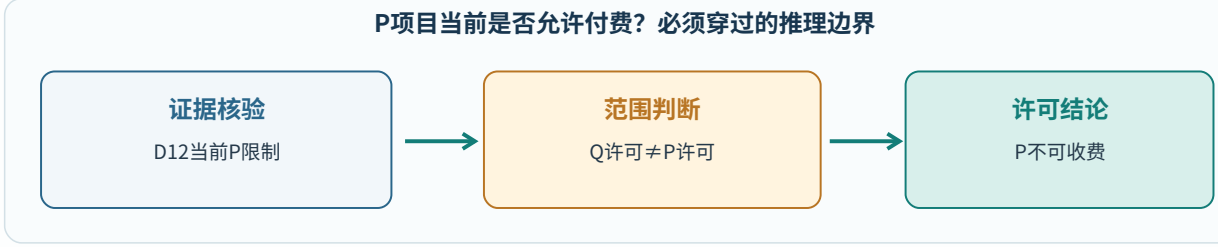


图15：正确处理项目收费许可需要证据、范围与有效权限三关。

7.6 第六步：把本轮新事件写入记忆

可以保存“D30用户询问P收费许可，系统核对后回答P仍未获授权，没有调用收费工具”；不应写成“D30 P实验继续”或“D30已批准”。错误来源的警示可以作为诊断记录保留，但要明确它是反例还是当前规则。

八、典型失效机制：不要把所有错误都归结为‘记忆差’



图12：结果错误可能来自证据、规划和执行等多个环节。

故障层	可以观察到什么	最可能的诊断起点
记忆写入	D12从未被正确保存	查原始事件与写入流水线
检索召回	D12没有返回	查查询、过滤、索引与Top-K
语义理解	D12已返回但模型认为是建议	查负向条件和来源角色的理解
时间与范围	Q的例外被误认为适用P	查项目ID、有效期、版本替代规则
规划	结论正确，却规划了付费步骤	查计划是否约束满足、前置条件
控制与工具	模型越权建议被实际执行	查控制器与实际权限检查
环境核验	工具响应200，但状态未改变	查异步任务、验收接口与业务状态
反馈写回	把“建议/尝试”记成“完成”	查日志、结果与摘要是否混写

这里可能存在**多重故障**。比如D12既未被检索、控制器也未阻断收费，不能只报一个“记忆召回失败”。要判断哪一层对结果具有必要或充分的影响，通常需要保持其他条件可比的分析，而不是凭自然语言轨迹猜测。

九、为什么整个系统的可靠性可能远低于每个部件看起来的可靠性

9.1 一个五道关口的教学概率例子

假设一个虚构场景中，按实际到达上一道关口且上一道已成功为条件，五道关口的条件成功率分别是：

- 找到正确证据：95%；
- 正确理解证据：90%；

- 做出合规计划：95%；
- 工具按预期执行：90%；
- 最终状态正确验收：95%。

那么**五关全部成功**的联合概率由条件概率链式法则得到：

$$P(\text{全链成功}) = 0.95 \times 0.90 \times 0.95 \times 0.90 \times 0.95 \approx 0.6945 = 69.45\%$$

这不是某种Agent的真实实验，也不是对未来成功率的预测。特别注意：上面乘的是**顺序条件成功率**，因此无需再额外假设五个事件独立；如果只有五项互不限定条件的**边际成功率**，直接相乘就会引入独立性假设，不能随意采用。

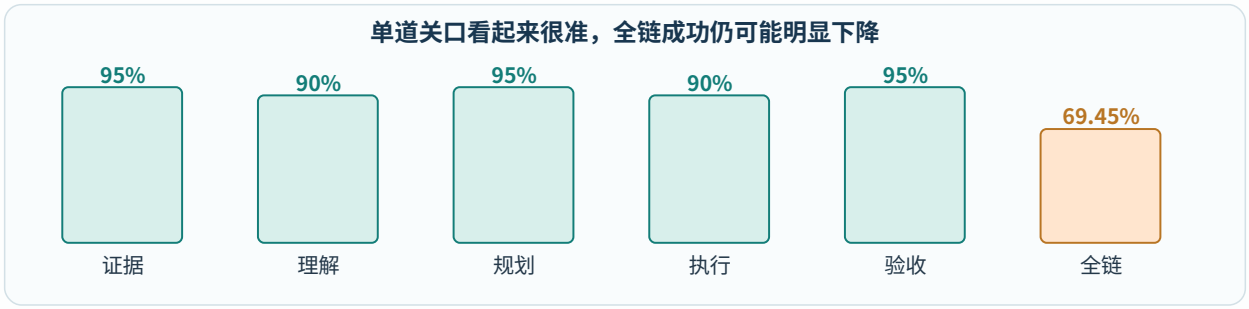


图13：五道关口与整个链条的教学概率对比（非实验结果）。

9.2 优化哪一关最值得?

例如仅把第二关“正确理解”从90%提升到97%，其余条件不变，联合成功率约为：

$$0.95 \times 0.97 \times 0.95 \times 0.90 \times 0.95 \approx 74.85\%$$

提升约5.40个百分点。实际应优先关注哪一关，还取决于该关的成本、错误严重性、条件相互作用与真实任务分布，不能只依据一套课堂概率机械选择。

9.3 质量与延迟也有权衡

虚构的单轮总延迟可拆为：记忆查询90ms + 回源核验80ms + LLM推理1,200ms + 控制器检查30ms + 工具执行300ms + 环境核验100ms = **1,800ms**。如果为了降低延迟删掉100ms环境核验，总时间只减少约5.6%，却可能失去区分“调用返回”与“任务真正完成”的关键证据。这个例子强调：**应评估全链质量—时间—成本，而不是只看某一组件的速度。**

十、三种架构方案：简单提示、证据结构化与执行硬约束

10.1 方案A：文本拼接型

检索Top-K片段→拼到Prompt→模型输出答案或动作。实现简单，适合低风险文本问答和概念验证；但来源、时间、作用域和授权依赖模型自行理解。即便Prompt写有“请勿违规”，也不是硬性执行阻断。

10.2 方案B：结构化证据型

先把候选整理成带`project_id`、`source_role`、`valid_time`、`evidence_id`的记录，逐项做证据核验后再生成答案。它通常更适合对当前事实和历史溯源要求较高的任务，但也会增加抽取、更新和维护成本。

10.3 方案C：控制器硬约束型

在B基础上，真实工具调用之前还做独立权限检查、预算检查、参数合法性检查和后续状态核验。它不是“让LLM在Prompt中更加小心”，而是用控制器提供**不能仅靠文字绕过**的执行规则。注意这只描述系统设计思想，具体规则仍会受实现、权限基础设施和威胁模型限制。



图14：从文本拼接到硬性控制器，是任务条件驱动的架构演进。

选择哪种架构应由任务要求决定：**回答一般知识问题**未必需要复杂硬约束和审计；**能够花钱、删除、发送或改变外部状态的Agent**则需要更严格的执行边界。构造越来越复杂的记忆系统不是天然科学创新，只有当复杂性解决了清晰、可检验的问题，才能证明其必要性。

十一、研究如何理解这一问题：读懂代表性工作，而不是误解论文

11.1 ReAct：推理与行动交替

ReAct提出在行动轨迹中交替生成推理和任务特定动作，动作又能从外部来源或环境获取观察信息。这解释了为什么工具反馈可能纠正纯文本推理的错误。[R1] 但是“能调用工具”与“已具备不可绕过的授权控制/状态核验”是两个不同命题。

11.2 Generative Agents：记忆影响规划

Generative Agents将过去经历以自然语言形式保存，并通过动态检索与反思支持代理行为规划。[R2] 这证明了记忆不仅用于问答；但从某个模拟世界中的可相信行为出发，不能直接推导真实科研Agent的费用管理、权限与环境执行一定可靠。

11.3 Reflexion：经验反馈影响后续尝试

Reflexion用自然语言反思与情景记忆引导后续决策，而不是每次依赖模型权重更新。[R3] 其价值在于说明**试错反馈也可以被保存、再利用**；风险是如果反馈本身不可靠，反思也可能把错误经验延续下去。

11.4 LongMemEval：索引、检索与阅读应分开评估

LongMemEval把长期记忆系统分为索引（indexing）、检索（retrieval）和阅读（reading）几个阶段，并评估信息提取、跨会话推理、时间推理、知识更新与弃答能力。[R5] 这给我们提供了重要分析框架：一个系统不是“召回更多”就必然“回答更好”，还要看读取环节是否准确处理时间和冲突。

11.5 长上下文位置偏差与解释可信性

Lost in the Middle显示在部分长上下文任务里，相关证据位置影响性能。[R4] Turpin等关于Chain-of-Thought的研究则指出，模型写出的推理解释可能无法忠实揭示真正影响答案的机制。[R7] 因此，科研分

析中可以把**模型输出理由**作为可观察文本，而不能把它自动认作内部因果证明。若要证明某条记忆改变了行为，需要可控制的比较和独立观察。

11.6 AgentBench和LangGraph提供什么视角？

AgentBench强调在交互环境中对LLM Agent进行多维评估，可帮助我们理解“动作是否达成环境目标”的概念。[R6] LangGraph的官方持久化文档区分线程检查点与跨线程Store，说明任务执行状态与长期信息并非一回事。[R8] 这些来源的任务、假设与实现差异很大，不应把其结果简单加在一起当成一个统一的记忆成功率。

十二、十二个零基础最容易误解的地方

1. **“找到了记忆=想明白了。”** 错。召回只证明候选可见，不证明理解。
2. **“检索分数最高的记忆最值得相信。”** 错。相关性与可信、有效和授权是不同维度。
3. **“模型计划了一件事=已经做了。”** 错。计划是候选行动序列。
4. **“工具调用文本已经输出=工具已经运行。”** 错。可能被控制器拒绝或根本没有发送。
5. **“HTTP 200一定意味着任务完成。”** 错。只证明某个接口层的响应，业务结果需核验。
6. **“HTTP 504说明工具肯定没有造成任何改变。”** 错。超时与下游完成状态可分离。
7. **“把历史全部塞给模型就能避免决策错误。”** 错。过量信息反而可能干扰。
8. **“最近一条记录总能覆盖旧记录。”** 错。先核实体、作用域、时间和替代关系。
9. **“模型自己写出理由就等于真实因果解释。”** 错。可读解释与内部机制可信性不同。
10. **“Reflexion等同于每次试验后重新训练权重。”** 错。核心机制是语言反馈与反思记忆。
11. **“有外部检索就不需要规划与控制器。”** 错。三个模块处理不同任务。
12. **“发生失败就说明记忆架构不好。”** 错。还可能是工具、执行器、环境或评分器错误。

十三、理论思考题：先自己推导，再看答案

题1｜分层定位。 D12的限制已经返回给模型，模型却把它说成“建议”。这是什么层的错误？为什么不能说“召回失败”？

题2｜作用域判断。 E12约束P，E21仅批准Q一次。D30是否允许P付费？需要哪些字段做证据？

题3｜观察边界。 模型输出`run_trial(P,100)`，但控制器记录`DENIED`，实际工具日志无请求。应怎样描述执行状态？

题4｜工具回执。 工具返回HTTP 200，随后状态查询显示未产生预期结果。哪些判据已满足，哪些未满足？

题5｜超时重复。 写入类工具返回504。为什么不能立即无条件重试？请指出最少两种核查方式。

题6｜上下文计算。 总预算8,192，输入控制1,280、本轮1,200、证据1,800、工具680、输出保留1,800。还余多少Token？是否必须全部用于记忆？

题7｜链式概率。 条件成功率为0.95、0.90、0.95、0.90、0.95，五关都成功的概率约多少？直接相乘需要独立性吗？

题8｜局部改进。 把第二关条件成功率从0.90提升为0.97，其余不变，联合成功率约多少？提升多少个百分点？

题9｜经验真实性。 D09日志504，D11 AI推测“模型差”，D18工具日志显示网关超时。三条信息应怎样写进记忆？

题10｜架构选择。 一个只能检索历史论文并回答“曾讨论过什么”的Agent，与一个能实际支付API费用的Agent，为什么需要不同的控制措施？

题11｜解释可信性。 模型在输出中写“我遵守D12”，但Trace显示它仍提出了付费请求。这如何说明自然语言解释的局限？

题12｜科研评价。 某方法比基线多召回了20条记忆，但最终任务成功率没有改善。至少提出三种不同层次的解释，并指出怎样在概念上加以区分。

十四、理论题答案与推导

答1。 这是“阅读/语义理解”层的错误，也可能涉及角色/模态分类；因为D12已确实返回，召回的前提已经满足。需要检查模型从“暂停”解析出的命题、是否漏了否定、是否把研究者决定误归类为建议。

答2。 **不允许**。E21只适用Q，不能替代P限制。至少核`project_id`、来源主体`source_role`、主张类型（提议/许可/限制）、有效时间、是否被后续明确替代以及原文证据指针。

答3。 只能说“模型提出候选付费动作，控制器拒绝，未观察到工具请求”。不能声称已执行，更不能说已产生费用。如果没有完整工具日志，只能按已有证据描述其范围，不能虚构未观察到的结果。

答4。 某个接口阶段得到成功响应；业务目标未成功验收。应记录工具响应200与状态未达成的差异，检查异步任务、请求参数、状态延迟或部分成功，而不是用单一`success=true`覆盖。

答5。 写入可能已被下游接受，却没有及时返回回执。应通过幂等键与操作ID核对、读取权威状态、查看服务端日志或向用户确认风险；无约束重试可能导致重复支付、重复写入等副作用。

答6。 总占用1,280+1,200+1,800+680+1,800=6,760；剩余8,192−6,760=**1,432 Token**。不必用满，因为不相关证据会增加干扰、延迟和费用，且容量可用于后续工具观察或安全边际。

答7。 乘积为$0.95\times0.90\times0.95\times0.90\times0.95\approx0.6945$，约**69.45%**。因为每个概率在题意下是“前面都成功”的条件成功率，所以是链式法则，不额外要求独立；若题目仅提供各环节无条件准确率，就不能任意作同样计算。

答8。 $0.95\times0.97\times0.95\times0.90\times0.95\approx0.7485$，约**74.85%**，比69.45%提高约**5.40个百分点**。课堂例子不代表真实系统提升幅度。

答9。 D09写成“观察：工具HTTP 504，关联请求ID”；D11写成“AI候选解释：模型能力问题，尚未核验”；D18写成“新诊断：按网关日志归因于代理超时，指向对应日志”。同时建立D18对D11的纠错或替代关系，不能把猜测伪装成工具原始证据。

答10。 历史问答通常主要涉及证据检索、引用与时序准确；付费Agent还涉及真实执行权限、预算、参数与环境反馈。模型阅读记忆可能犯错，必须在执行控制层实施有效授权约束，并在执行后核对实际效果。

答11。 可读输出与实际行动提议不一致；语言解释不能自动证明执行策略已遵守限制，应核对可观察的工具Trace和真实控制器状态。输出理由是证据之一，不是内部因果机制或成功的最终证明。[R7]

答12。 可能①新增的20条是过时、无关或重复内容（候选质量问题）；②模型没有正确使用召回的有效证据（阅读/推理问题）；③计划仍违反前置条件（规划问题）；④工具或环境失败（执行问题）；⑤评分器不敏感或有漏标（测量问题）。理论上可分别检查证据有效性、上下文阅读结果、行动计划、真实工具反馈及独立评分合同，不能因为一个终局数字就直接否定检索机制。

本讲总结与第25讲预告

把今天的知识压缩成一条可记忆、可推演的主线：

外部记忆先作为证据进入上下文；模型必须正确识别证据的身份、时间和作用域；规划器必须提出满足约束的下一步；控制器决定能否真正执行；环境反馈决定任务有没有完成；最后只有经过标注和核验的结果才能可靠地写回长期记忆。

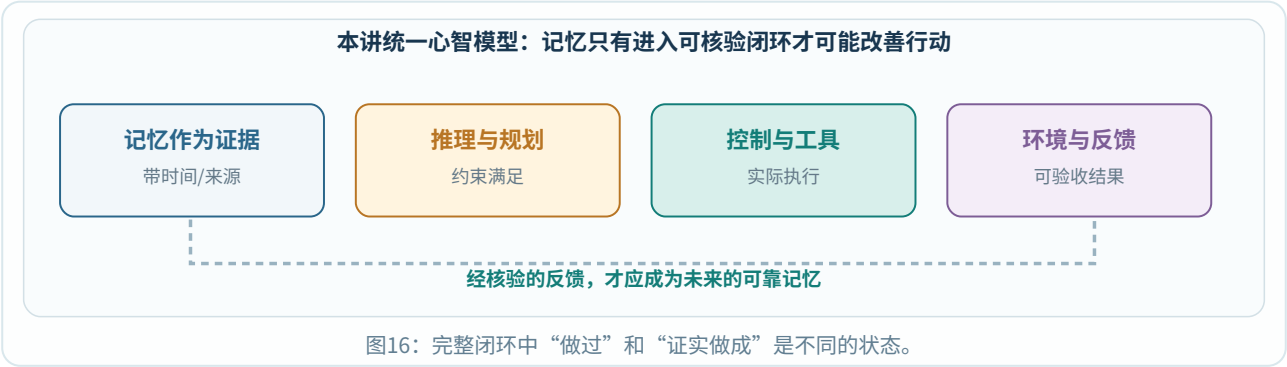


图16：完整闭环中“做过”和“证实做成”是不同的状态。

本讲到这里解决“记忆如何用于行动”这个完整闭环。**第25讲**将进一步讨论记忆更新、遗忘和动态演化中的维护问题：什么情况下应该修订旧记忆、哪些信息应当保持历史、怎样避免同一条错误在系统中长期扩散。

参考文献与来源（核对至2026-10-19）

[R1] Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. https://openreview.net/forum?id=WE_vluYUL-X

[R2] Park, J. S., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. https://doi.org/10.1145/3586183.3606763

[R3] Shinn, N., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[R4] Liu, N. F., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL 12:157–173. https://aclanthology.org/2024.tacl-1.9/

[R5] Wu, D., et al. (2025). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html

[R6] Liu, X., et al. (2024). AgentBench: Evaluating LLMs as Agents. ICLR 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/e9df36b21ff4ee211a8b71ee8b7e9f57-Abstract-Conference.html

[R7] Turpin, M., et al. (2023). Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. NeurIPS 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/ed3fea9033a80fea1376299fa7863f4a-Abstract.html

[R8] LangGraph Documentation. Persistence: Checkpointers vs. Stores. 官方开发文档（2026-10-19查阅）. https://langchain-ai.github.io/langgraph/concepts/durable_execution/

[R9] Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf

出处界线： [R1]—[R9]用于支持论文确实讨论的主题与研究结论。正文中的“五关模型”、概率数字、P/Q项目、Token预算、工具日志、控制器状态机和全部示意图，均由本课程为教学自行构造，不代表论文作者使用了相同的配置或报告了相同的数值。